

# MVC Project: Multilayer Perceptron for Student Performance Prediction

Muhammad Ahmed Butt

National University of Computer and Emerging Sciences (FAST-NU),  
Islamabad, Pakistan  
25i3010@nu.edu.pk  
<https://github.com/MuhammadAhmedButt25i3010/mvc-mlp-25i3010>

**Abstract.** This report presents a comprehensive implementation and analysis of a Multilayer Perceptron (MLP) starting from mathematical derivation to computer application. This project is divided into two main phases: a manual derivation phase and a programmed execution phase (Python). In the initial phase, a 2-2-2-1 neural architecture was manually constructed to predict student performance. This involved performing step-by-step forward and backward propagation for multiple data samples. Few key insights from the manual are in regards to the vanishing gradient problem being identified in early layers and SGD vs Mini Batch effectiveness.

The second phase transitions these manual concepts into a functional Python implementation. By using the NumPy library, the manual neuron-by-neuron calculations were converted into vectorized matrix operations, allowing the model to scale efficiently. This implementation was specifically tested on the MNIST dataset of handwritten digits. Beyond basic gradient descent, more advanced optimization techniques like Momentum and Adam were integrated to observe their impact on convergence speed and accuracy. The final results are documented through loss curves and performance metrics, demonstrating how a from-scratch MLP can effectively learn complex patterns in high-dimensional image data.

**Keywords:** Multilayer Perceptron · Backpropagation · Gradient Descent · MNIST · Neural Networks

## 1 Introduction

### 1.1 Multilayer Perceptron (MLP)

An MLP is a type of feedforward artificial neural network composed of an input layer, one or more hidden layers, and an output layer. Each neuron in a layer is fully connected to neurons in the next layer, with weights determining the strength of these connections and biases allowing flexibility in activation thresholds.

**Project Goal** The goal of this project is to use these networks for two different tasks: predicting student success through manual calculations and performing digit classification using Python.

## 2 Background

The MLP architecture used in the manual phase consists of an input layer, two hidden layers, and an output layer (2-2-2-1). Each neuron  $j$  in layer  $l$  calculates its output as:

$$a_j^{(l)} = \sigma \left( \sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)} \right) \quad (1)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2)$$

where  $\sigma$  is the Sigmoid activation function. The error is measured using Mean Squared Error (MSE):

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (3)$$

### 2.1 Vanishing Gradient Problem

During the manual backpropagation phase, the gradients for the earlier layers ( $w_1-w_4$ ) were noticeably smaller than those of the output layer. This is because each layer multiplies by a sigmoid derivative, which is always  $\leq 0.25$ , so the further back you go the smaller the gradient gets. This means the earlier weights barely update compared to the output weights, which is a known issue with sigmoid-based networks referred to as the vanishing gradient problem.

## 3 Implementation

### 3.1 Sigmoid Activation Function

The sigmoid function and its derivative were implemented as follows:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (4)$$

### 3.2 Forward Pass

The forward pass computes the weighted sum and activation for each layer sequentially. For a batch of  $m$  samples, the computation is formulated using matrix multiplication:

$$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)} \mathbf{W}^{(l)} + \mathbf{b}^{(l)}, \quad \mathbf{A}^{(l)} = \sigma(\mathbf{Z}^{(l)}) \quad (5)$$

All intermediate activations  $\mathbf{A}^{(1)}$ ,  $\mathbf{A}^{(2)}$ ,  $\mathbf{A}^{(3)}$  are stored for use in backpropagation.

### 3.3 Backpropagation

Gradients are computed layer by layer from output to input using the chain rule. The delta at each layer is:

$$\delta^{(3)} = -2(\mathbf{Y} - \mathbf{A}^{(3)}) \odot \sigma'(\mathbf{A}^{(3)}) \quad (6)$$

$$\delta^{(l)} = (\delta^{(l+1)} \mathbf{W}^{(l+1)T}) \odot \sigma'(\mathbf{A}^{(l)}) \quad (7)$$

Weight gradients are computed as:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \frac{1}{m} \mathbf{A}^{(l-1)T} \delta^{(l)}, \quad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \frac{1}{m} \sum_{i=1}^m \delta^{(l)(i)} \quad (8)$$

### 3.4 Weight Update

Parameters are updated using gradient descent:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}, \quad \mathbf{b} \leftarrow \mathbf{b} - \eta \frac{\partial L}{\partial \mathbf{b}} \quad (9)$$

### 3.5 Training Procedure

Training used mini-batch gradient descent with the following procedure per epoch:

1. Shuffle training data
2. Split into mini-batches of size  $B = 32$
3. For each batch: forward pass  $\rightarrow$  compute loss  $\rightarrow$  backpropagate  $\rightarrow$  update weights
4. Record a epoch loss over the full training set

## 4 Experiments

### 4.1 Dataset

The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0–9), each of size  $28 \times 28$  pixels. We used 60,000 images for training and 10,000 for testing. Each image was flattened into a vector of 784 values and normalised to  $[0, 1]$  by dividing by 255. Labels were converted to one-hot vectors of length 10. As stated in the project manual.

### 4.2 Parameters

#### 4.3 Manual Trace (Tasks 1–6)

The manual derivation was performed on a 2-2-2-1 network with assigned weights:  $w_1 = -0.5$ ,  $w_2 = -0.41$ ,  $w_3 = -0.86$ ,  $w_4 = 0$ ,  $w_5 = 0.11$ ,  $w_6 = 0.58$ ,  $w_7 = 0.74$ ,  $w_8 = -0.09$ ,  $w_9 = -0.75$ ,  $w_{10} = -0.2$ ,  $b^{(1)} = 0.26$ ,  $b^{(2)} = -0.26$ .

The forward pass for sample  $s^{(1)}$  ( $x_1 = 0.2$ ,  $x_2 = 0.8$ ,  $y = 1$ ) produced  $\hat{y} = 0.3764$ , giving an initial MSE loss of  $L = 0.3050$  across all three samples. After one gradient descent update with  $\eta = 0.1$ , the loss decreased to 0.3027, confirming correct backpropagation.

**Table 1.** Training parameters

| Parameter              | Value                 |
|------------------------|-----------------------|
| Learning rate $\eta$   | 0.1                   |
| Epochs                 | 20                    |
| Batch size $B$         | 32                    |
| Weight initialization  | Uniform $[-0.5, 0.5]$ |
| Bias initialization    | 0                     |
| Hidden layer 1 neurons | 128                   |
| Hidden layer 2 neurons | 64                    |
| Output neurons         | 10                    |
| Activation             | Sigmoid               |
| Loss function          | MSE                   |

**Table 2.** Mini-Batch MSE Reduction (Tasks 5)

| Epoch | Batch | Samples            | MSE Loss |
|-------|-------|--------------------|----------|
| 1     | 1     | $s^{(1)}, s^{(2)}$ | 0.3022   |
| 1     | 2     | $s^{(3)}$          | 0.3015   |
| 2     | 1     | $s^{(3)}, s^{(1)}$ | 0.2988   |
| 2     | 2     | $s^{(2)}$          | 0.2964   |
| 3     | 1     | $s^{(2)}, s^{(3)}$ | 0.2941   |
| 3     | 2     | $s^{(1)}$          | 0.2922   |

#### 4.4 Optimizer Comparison (Task 6C)

Three optimizers were compared over 10 iterations using sample  $s^{(1)}$  only (SGD-style). Momentum used  $\beta = 0.9$  with the scaled variant  $v_t = \beta v_{t-1} + (1 - \beta)g$ .

Plain GD outperformed Momentum and NAG in early iterations because the scaled momentum variant of GD initialises velocity at zero, meaning only 10% of the gradient is used initially. Momentum would surpass plain GD given more iterations. Therefore, this result is within expectations.

#### 4.5 Python Results

**Loss Curve** The training loss decreased consistently from 0.0296 at epoch 1 to 0.0069 at epoch 20, confirming that the network learned effectively.

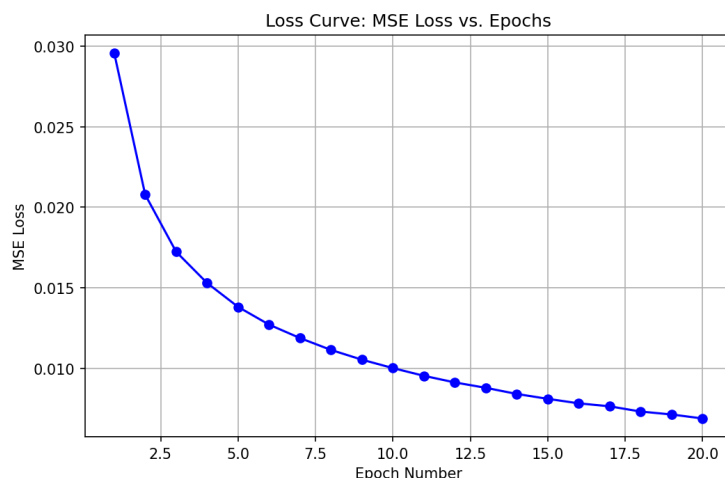
**Test Accuracy** The final classification accuracy on the 10,000 test images was:

$$Accuracy = 95.54\% \quad (10)$$

#### Sample Predictions

**Table 3.** MSE Loss Comparison Across Optimizers

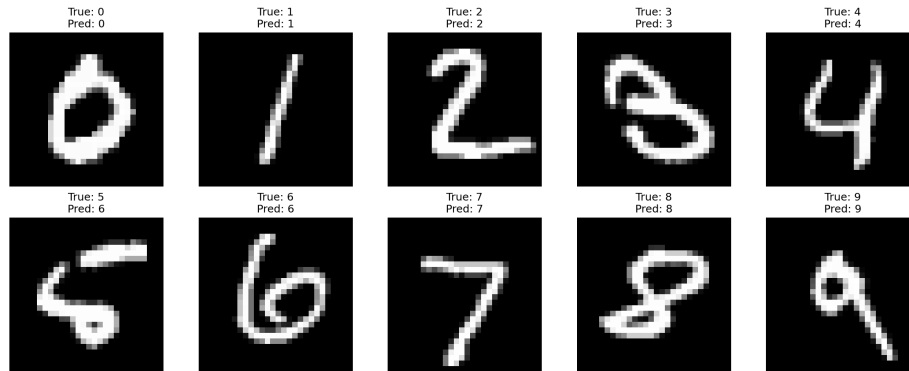
| Iteration | Plain GD | Momentum | NAG    | Fastest  |
|-----------|----------|----------|--------|----------|
| 1         | 0.3888   | 0.3888   | 0.3888 | Tie      |
| 2         | 0.3837   | 0.3883   | 0.3883 | Plain GD |
| 3         | 0.3787   | 0.3873   | 0.3873 | Plain GD |
| 4         | 0.3738   | 0.3859   | 0.3859 | Plain GD |
| 5         | 0.3690   | 0.3842   | 0.3842 | Plain GD |
| 6         | 0.3642   | 0.3821   | 0.3821 | Plain GD |
| 7         | 0.3596   | 0.3798   | 0.3798 | Plain GD |
| 8         | 0.3550   | 0.3772   | 0.3772 | Plain GD |
| 9         | 0.3506   | 0.3743   | 0.3744 | Plain GD |
| 10        | 0.3462   | 0.3713   | 0.3714 | Plain GD |

**Fig. 1.** MSE Loss vs Epoch Number over 20 training epochs

## 5 Discussion

At the start, the randomly initialised weights gave predictions around 0.38 for all three samples regardless of the input values, which shows the network was essentially guessing. This confirmed that training is necessary for the weights to actually learn anything useful. The vanishing gradient problem was also visible in the numbers; gradients for  $w_1$ – $w_4$  were around  $10^{-3}$  while the output layer gradients were much larger, meaning the earlier layers were barely updating compared to the output layer.

For the optimizer comparison, plain GD actually won across all 10 iterations. This occurred due to the scaled momentum formula ( $v_t = \beta v_{t-1} + (1 - \beta)g$ ), the velocity starts at zero and only 10% of the gradient is used in the first few steps. So momentum needs more iterations to build up before it becomes useful. With more iterations or a larger dataset it would eventually overtake plain GD.



**Fig. 2.** Sample predictions — one image per digit class (0–9)

The Python implementation ended up hitting 95.54% accuracy on MNIST. Given sigmoid and MSE are not the ideal choices for a classification problem, this was an ideal occurrence. Swapping sigmoid for ReLU and MSE for cross-entropy would likely get it above 98%.

## 6 Conclusion

This project covered the full process of building a neural network from scratch; starting from manually computing forward passes and gradients on paper, all the way to training on MNIST in Python. The manual phase made it clear how back-propagation actually works and where problems like vanishing gradients come from. The Python phase showed how the same logic scales up using vectorised matrix operations. The main takeaways were understanding why sigmoid networks struggle with vanishing gradients, why momentum needs time to become effective, and how a relatively simple and novice MLP can still achieve strong results on a real dataset.

## References

1. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* **323**, 533–536 (1986).
2. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. of the IEEE* **86**(11), 2278–2324 (1998).
3. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).